

Build Brief: expense-verification-pipeline

Target path: `C:\Users\msell\OneDrive\aialchemyrepos\expense-verification-pipeline`

Input data files (already placed at repo root):

- `C:\Users\msell\OneDrive\aialchemyrepos\expense-verification-pipeline\sample_expenses.xlsx` (now includes a `department` column)
- `C:\Users\msell\OneDrive\aialchemyrepos\expense-verification-pipeline\sample_policy_manual.docx`

Paste this whole document to Claude Code as the initial build instruction.

Objective

Build a gated, multi-agent pipeline that checks expense reports against a company spending policy, loads verified results into Snowflake, and exposes a Streamlit dashboard (deployable as a native Streamlit-in- Snowflake app, or standalone at any URL) showing individual employees, reimbursement amounts, and department-level breakdowns with charts.

Core principle to enforce everywhere: **gates check artifacts and re-computed values, never LLM output text or a "status: success" flag.** This now extends to the Snowflake load step: the load is only considered successful once an independent row-count query against Snowflake confirms it — never because the loader function returned without an exception.

Repo Structure

expense-verification-pipeline/

sample_expenses.xlsx # real input spreadsheet (already present, has `department`)

sample_policy_manual.docx # real policy manual (already present)

src/

expense_pipeline/

__init__.py

cli.py # entry point: expense-verify run <spreadsheet> <policy>

ingestion.py # Stage 1: parse .xlsx spreadsheet

policy_parser.py # Stage 2: parse .docx policy manual into structured rules

checker.py # Stage 3: LLM-assisted compliance reasoning

verifier.py # Stage 4: pure-code independent re-derivation

approver.py # Stage 5: finalize, split into approved / needs_review

snowflake_loader.py # Stage 6: load verified results into Snowflake

gates.py # hardcoded precondition/postcondition checks

audit.py # writes JSON audit trail per run

schemas.py # dataclasses / pydantic models for all artifacts

snowflake_conn.py # connection helper, reads credentials from env only

dashboard/

 app.py # Streamlit app — queries Snowflake directly

 queries.py # named SQL queries used by the dashboard

data/

 policy_rules.yaml # structured rules parsed FROM sample_policy_manual.docx

sql/

 001_create_tables.sql # EXPENSE_VERDICTS + RUN_SUMMARY DDL

audit/ # audit JSON output (gitignored)

reports/ # human-readable run reports (gitignored)

tests/

test_ingestion.py

test_policy_parser.py

test_checker.py

test_verifier.py

test_gates.py

test_snowflake_loader.py

test_end_to_end.py

.env.example # documents required Snowflake env vars, no real values

CLAUDE.md # policy for AI agents working in this repo

README.md

.gitignore

pyproject.toml

Data Contracts

sample_expenses.xlsx (Stage 1 input — real file, repo root)

Sheet **expenses**, columns: **report_id**, **employee**, **department**, **date**, **category**, **amount**, **currency**, **receipt_attached**, **notes**

Categories present: **meals**, **travel**, **lodging**, **software**, **office_supplies**, **client_entertainment**. Departments present: **Sales**, **Engineering**, **Finance**, **Marketing**.

Parsing: **openpyxl** or **pandas.read_excel()**. Do not hardcode a row count — **ingestion.py** reads whatever is actually in the sheet.

`sample_policy_manual.docx` (Stage 2 input — real file, repo root)

Same as before — Section 4 contains the authoritative category-limits table, extract via `python-docx`.

`data/policy_rules.yaml` (Stage 2 output)

Unchanged from prior spec — one entry per category with `daily_limit`, `receipt_required_above`, and (for software / client_entertainment) `requires_manager_approval_above`.

Per-row verdict schema (Stages 3 and 4)

```
{  
  
  "report_id": "EXP-0001",  
  
  "verdict": "approved | flagged",  
  
  "reasons": ["exceeds daily_limit for category", "missing receipt"],  
  
  "rule_citations": ["meals.daily_limit", "meals.receipt_required_above"]  
}
```

Stage Specs (Stages 1–5 unchanged from prior brief)

Stage 1 — Ingestion Agent (`ingestion.py`)

Reads `sample_expenses.xlsx`. Gate re-opens the file itself and confirms row count matches. (See prior brief for full detail — unchanged.)

Stage 2 — Policy Parser Agent (`policy_parser.py`)

Reads `sample_policy_manual.docx`, writes `data/policy_rules.yaml`. Gate confirms every category in the expense data has a rule entry.

Stage 3 — Compliance Checker Agent (`checker.py`)

LLM-assisted judgment on ambiguous rows. Gate confirms row-count parity and every row has a verdict.

Stage 4 — Independent Verifier (`verifier.py`)

Pure code, no LLM, re-derives verdicts from the raw spreadsheet and raw policy YAML independently of the Checker's output. Gate compares the two sets of verdicts; disagreements are collected, not raised.

Stage 5 — Approver Agent (`approver.py`)

Splits rows into approved / flagged / needs_human_review based on Checker+Verifier agreement, per prior spec.

Stage 6 — Snowflake Loader Agent (`snowflake_loader.py`) — NEW

- Takes the final, approver-resolved row set (approved + flagged + needs_human_review, each tagged with its `final_status`) and loads it into the `EXPENSE_VERDICTS` table (see DDL below), tagged with the `run_id`.
- Also loads one row into `RUN_SUMMARY` with the run's aggregate counts.
- **Gate to "load complete":** `gates.gate_snowflake_load_verified()` does **not** trust the loader's return value. It independently runs `SELECT COUNT(*) FROM EXPENSE_VERDICTS WHERE run_id = %s` against Snowflake and confirms the count equals the number of rows the Approver produced for that run. Raises `GateFailure` on mismatch — this is the same independent-verification pattern as Stage 4, applied to the database write instead of a business-rule verdict.

Snowflake Schema (`sql/001_create_tables.sql`)

```
CREATE TABLE IF NOT EXISTS EXPENSE_VERDICTS (
```

run_id	STRING,
report_id	STRING,
employee	STRING,
department	STRING,
expense_date	DATE,
category	STRING,
amount	NUMBER(10,2),

```
currency          STRING,
receipt_attached  BOOLEAN,
notes            STRING,
checker_verdict   STRING,
checker_reasons   STRING,
verifier_verdict  STRING,
verifier_reasons  STRING,
final_status      STRING, -- approved | flagged | needs_human_review
loaded_at         TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);
```

```
CREATE TABLE IF NOT EXISTS RUN_SUMMARY (
```

```
run_id           STRING,
run_timestamp     TIMESTAMP_NTZ,
rows_processed    NUMBER,
approved_count    NUMBER,
flagged_count     NUMBER,
needs_review_count NUMBER,
loaded_at         TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);
```

Snowflake Connection (`snowflake_conn.py`)

- Reads connection details **only from environment variables** — never hardcoded, never committed: `SNOWFLAKE_ACCOUNT`, `SNOWFLAKE_USER`, `SNOWFLAKE_PASSWORD` (or `SNOWFLAKE_PRIVATE_KEY_PATH` for key-pair auth), `SNOWFLAKE_WAREHOUSE`, `SNOWFLAKE_DATABASE`, `SNOWFLAKE_SCHEMA`, `SNOWFLAKE_ROLE`.
 - `.env.example` documents each variable with a placeholder, no real values. `.env` itself is gitignored.
 - Use `snowflake-connector-python` for the pipeline's write path, and `snowflake-snowpark-python` (or the connector) for the dashboard's read path — whichever the Streamlit-in-Snowflake runtime provides natively when deployed there.
-

Dashboard (`dashboard/app.py`) — NEW

Built in Streamlit so it can run two ways with the same codebase:

1. **Native Streamlit in Snowflake** — deployed directly inside your Snowflake account (Snowsight → Streamlit apps), querying `EXPENSE_VERDICTS` / `RUN_SUMMARY` via the built-in Snowpark session — no external credentials needed at all in that mode.
2. **Standalone, any URL** — `streamlit run dashboard/app.py` on any host (local, EC2, Streamlit Community Cloud, etc.), using `snowflake_conn.py` and the env vars above to connect out to the same Snowflake tables.

`dashboard/app.py` should detect which context it's running in (a Snowpark session is auto-available in Streamlit-in-Snowflake; otherwise fall back to `snowflake_conn.py`) and use whichever is available — write one connection-getter function that both branches call the same `queries.py` functions against.

Required views/sections

1. **Individual expense table** — every row from `EXPENSE_VERDICTS` for the selected run: employee, department, date, category, amount, receipt status, final_status. Filterable by department, category, and final_status. Sortable by amount.
2. **Department breakdown**
 - Bar chart: total reimbursed amount by department
 - Bar or stacked chart: count of approved / flagged / needs_review by department
3. **Category breakdown**
 - Pie or bar chart: total spend by category

4. **Run summary header** — pulled from `RUN_SUMMARY`: rows processed, approved/flagged/needs_review counts, run timestamp. Let the user pick which `run_id` to view if more than one run has been loaded.
5. **Needs-review callout** — a distinct, visually separated table for rows with `final_status = 'needs_human_review'`, since those are the ones requiring a human decision, not just informational flags.

Use Streamlit's native chart functions (`st.bar_chart`, `st.plotly_chart` if richer interactivity is wanted) — no need for a separate JS frontend.

`gates.py` Design Rules (unchanged, extended)

- Every gate takes a filesystem path, a freshly re-computed value, or (new) a freshly re-queried database value — never a status string or LLM text, and never the writer's own return value.
 - `GateFailure` halts the run and logs to the audit JSON, except the Stage 4→5 verdict-disagreement gate, which routes to human review instead of halting.
 - One pytest per gate, including `test_snowflake_loader.py` covering both the passing case (count matches) and failing case (mocked mismatch) for `gate_snowflake_load_verified()`.
-

CLI

```
expense-verify run sample_expenses.xlsx sample_policy_manual.docx --load-to-snowflake
```

Without `--load-to-snowflake`, the pipeline stops after Stage 5 and writes local report CSVs only (useful for offline testing without Snowflake credentials configured). With the flag, Stage 6 runs and the CLI prints the Snowflake load confirmation (independently verified row count) alongside the existing summary.

Build Order for Claude Code

1. Scaffold repo structure, `pyproject.toml`, `.gitignore`, `.env.example`. Confirm `sample_expenses.xlsx` and `sample_policy_manual.docx` are already present at repo root before proceeding.

2. Implement `schemas.py` (dataclasses for all artifacts, including `department` on the expense row model).
 3. Implement `ingestion.py` (reads the real `.xlsx`, including `department`) + its gate + tests.
 4. Implement `policy_parser.py` (reads the real `.docx`) + its gate + tests.
 5. Implement `checker.py` (LLM-assisted) + its gate + tests.
 6. Implement `verifier.py` (pure code, independent) + its gate + tests. Confirm it never reads Checker output before computing its own verdict.
 7. Implement `approver.py` + `audit.py` + `cli.py` (without Snowflake yet) and get the local-only path fully working and tested first.
 8. Write `sql/001_create_tables.sql`, `snowflake_conn.py`, and `snowflake_loader.py` + `gate_snowflake_load_verified()` + tests (mock the Snowflake connection for unit tests; a real integration test can be manual, gated behind actual credentials being present).
 9. Build `dashboard/app.py` + `queries.py`. Test locally first via `streamlit run dashboard/app.py` against a real or sandbox Snowflake database before attempting the Streamlit-in-Snowflake deployment.
 10. Write `test_end_to_end.py` covering the full local path (Stages 1–5) and, separately, a Snowflake-load test that can be skipped when credentials aren't present in the environment.
 11. Write `CLAUDE.md` and `README.md`, including a short "Deploying the dashboard" section covering both the Streamlit-in-Snowflake path (Snowsight → Streamlit apps → point at `dashboard/app.py`) and the standalone path (`streamlit run dashboard/app.py` plus the env vars needed).
-

Safety Rules to Bake In

- No destructive filesystem or database operations.
- No secrets/credentials committed anywhere — `.env` gitignored, `.env.example` has placeholders only.
- `audit/` and `reports/` in `.gitignore`.
- Verifier stage (Stage 4) must remain independently testable with zero LLM/network calls.
- Snowflake writes are additive (`INSERT`, tagged by `run_id`) — never `TRUNCATE/DELETE` existing run history without an explicit, separate cleanup command outside the normal pipeline run.